

Aardappel

- **Tijdslimiet:** 10 minuten voor het hele bestand
- **Omgeving:** één GPU (≈16 GB VRAM), geen internet
- **Grootte van de oplossing:** `solution.ipynb` ≤ 1 MB
- **Opslag:** 5 GB

Opdracht

Je vriend stelt voor een spel te spelen. Omdat hij van woordspellen houdt heeft hij een variatie van het beste spel ooit: galgje. Hij kiest als jury één verborgen woord uit een vaste woordenschat, en jij moet dit binnen hooguit 30 beurten vinden. In elke beurt vergelijkt de jury twee woorden en meldt welk woord qua betekenis dicht bij het verborgen woord ligt. Elk spel begint met het vaste paar `lamp` vs `potato`, omdat dit twee van de favoriete dingen van je vriend zijn. Vervolgens stelt je programma één nieuw woord voor. De winnaar van de vergelijking blijft behouden en wordt met je volgende voorstel vergeleken. Je wint een spel zodra je exact het verborgen woord voorstelt. Bij het vergelijken wordt geen onderscheid gemaakt tussen hoofdletters en kleine letters. Elk woord dat je voorstelt, moet in de woordenlijst `dataset/vocabulary.json` staan.

Er staat een volledig voorbeeld in `solution.ipynb` met het protocol en het laden van de data. Je kunt de klasse `PublicEmbeddingPlayer` aanpassen. Je programma wordt eenmaal geïnitieerd en speelt elk spel binnen één run; het protocol maakt aan het begin van elk spel een nieuwe `PublicEmbeddingPlayer` aan.

De jury

Je programma stuurt één JSON-object naar de jury en de jury antwoordt met één JSON-object.

Een uitgewerkt voorbeeld, waarbij het verborgen woord uitsluitend wordt getoond om het protocol uit te leggen:

```
Hidden word: shovel          Fixed opening: lamp vs potato

(Jury zegt) <- {"turn": 1, "winner_word": "potato", "verdict": "second", "word1": "lamp", "word2": "potato"}
(Programma antwoord) -> {"new_word": "rock"}
<- {"turn": 2, "winner_word": "rock", "verdict": "second", "word1": "potato", "word2": "rock"}
-> {"new_word": "hammer"}
<- {"turn": 3, "winner_word": "hammer", "verdict": "second", "word1": "rock", "word2": "hammer"}
-> {"new_word": "shovel"}          <- matches: game won
-> {"status": "win"}
```

De beurten zijn genummerd van 1 tot en met 30.

De opties voor `verdict` zijn `first`, wat betekent dat `word1` dichterbij ligt, `second`, wat betekent dat `word2` dichterbij ligt, of `same`, wat betekent dat beide woorden even dicht bij het verborgen woord liggen.

`winner_word` is het woord dat voor de volgende vergelijking behouden blijft. Bij een `same`-oordeel blijft het eerste woord behouden.

Dataset

Gedeeld door elke split:

- `dataset/vocabulary.json` — 1602 unieke woorden in kleine letters. Het verborgen woord is altijd een van deze woorden.
- `dataset/public_embeddings.npy` — float32, vorm (1602, 2560). Rij `i` correspondeert met woord `i` in de woordenschat. Dit zijn *publieke* embeddings; de jury gebruikt een andere, *private* representatie.

De splits zijn verzamelingen verborgen woorden:

Split	Woorden	Antwoorden	Gebruik deze om
<code>test_public</code>	120	✅ <code>dataset/test_public.json</code>	je oplossing uit te voeren en zelf te scoren
<code>test_leaderboard_a</code>	120	verborgen	live leaderboard
<code>test_leaderboard_b</code>	120	verborgen	eindrangschikking

Er is geen `train-split` — er wordt niets gefit op basis van gelabelde rijen.

Beschikbare modellen

Bij de opdracht worden twee vooraf getrainde embeddingmodellen geleverd die mogen worden gebruikt:

```
models/Qwen/Qwen3-Embedding-0.6B
[Qwen Embedding model card] (https://yastatic.net/s3/contest/ioai/3/01_Qwen_Qwen3-Embedding-0.6B_MODEL_CARD.html)
models/BAAI/bge-m3
[bge-m3 model card] (https://yastatic.net/s3/contest/ioai/3/02_BAAI_bge-m3_MODEL_CARD.html)
```

Beide moeten vanaf hun lokale pad worden geladen; een Hugging Face-hub-id zoals "BAAI/bge-m3" activeert een download en faalt, omdat de beoordeling offline plaatsvindt. Elke directory bevat een uitvoerbaar `example.py` dat de offline aanroep toont.

Beschikbare libraries: `numpy`, `torch`, `sentence-transformers`. Geen internet, geen downloads, geen andere packages.

Uitvoer

Geen. Dit is een interactieve opdracht: je oplossing schrijft geen antwoordbestand; ze communiceert met de jury via `stdin/stdout` (`input()`, `print(...)`) zoals hierboven beschreven.

Metriek

Een spel dat in beurt t wordt opgelost, krijgt een score van $1.0 - 0.02 \times \max(0, t - 10)$; een spel dat niet binnen 30 beurten wordt opgelost, krijgt een score van 0. Beurten 1–10 krijgen dus een score van 1.00, beurt 20 krijgt een score van 0.80 en beurt 30 krijgt een score van 0.60.

Je opdrachtscore is de gemiddelde spelscore $\times 100$, tussen 0.00 en 100.00.

De limiet van 10 minuten is één totaalbudget voor het opstarten, de voorbereiding en alle 120 spellen in de testset.

Indienen

1. Open `solution.ipynb`, bewerk `PublicEmbeddingPlayer` en voer alle cellen uit om te controleren of alles werkt.
2. Controleer dit (als je het heeeeeeel graag wil) lokaal: `python local_test.py solution.ipynb --limit 5`. De lokale jury gebruikt de *publieke* embeddings, dus de score daarvan is slechts een indicatie.
3. Sla `solution.ipynb` op.
4. Open het tabblad Git in de linkerbalk van JupyterLab.
5. Stage `solution.ipynb` (het +-pictogram ernaast).
6. Voer een commitbericht in en klik op Commit.
7. Klik op het wolkje met de omhoogwijzende pijl om te pushen.
8. Keer terug naar deze wedstrijdpagina en klik op Submit, waarbij het commitbericht overeenkomt met het bericht dat je hebt opgegeven.

Dien precies één bestand in, met de naam `solution.ipynb`, dat alle noodzakelijke voorbereidingen en inferentie omvat.